

1.1 The diffusion equation

Diffusion equation: $u_t = u_{xx}$

Dirichlet boundary condition: $u(t, 0) = u(t, 1) = 0$

Initial condition: $u(0, x) = g(x)$

with $g(x) = \sin^2(\pi x)$.

Method-of-lines discretization:

- A semi-discretization in space: $U(t)_j \approx u(t, x_j)$
- Discretization by central difference quotients, obtaining an ODEs system that we can write in a matrix - vector form: $\dot{U}(t) = T_{\Delta x} U(t)$
- A full discretization which discretizes the semi-discretization in time
→ Explicit Euler Method:

$$U^{n+1} = U^n + T_{\Delta x} U^n$$

- $[0, 1], \Delta x = \frac{1}{N+1}$

1.1 Toeplitz matrix

$T_{\Delta x}$ is the Toeplitz symmetric tridiagonal matrix that approximates $\frac{\partial^2}{\partial x^2}$ to the second order accuracy on the grid.

```
function T = Tdx(N,dx)

e = ones(N,1);
T = (diag(-2*e,0) + diag(e(1:end-1),1) + diag(e(1:end-1),-1)) / (dx^2);
end|
```

1.1

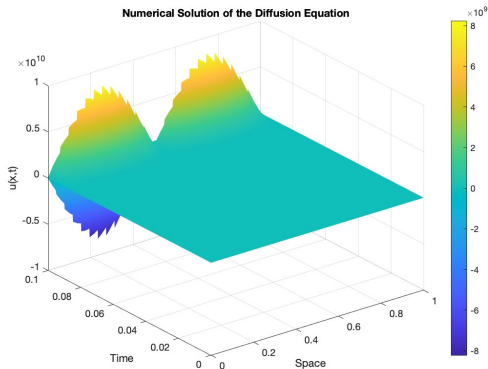
```
T = Tdx(N,dx); % matrix Tdx, apply it to the internal points.  
  
% store solutions at each time step for visualization  
% the solution is defined on N+2 points, including boundaries  
U = zeros(N+2,M+1);  
U(:,1) = u0;  
  
% time vector  
tt = linspace(0,tend,M+1);  
  
% explicit Euler time stepping:  
uold = u0;  
for m = 1:M  
    % apply explicit Euler step on the internal points  
    unew = eulerstep2(T,uold(2:end-1),dt);  
  
    % insert boundary conditions  
    ufull = [0; unew; 0];  
  
    % update  
    U(:,m+1) = ufull;  
    uold = ufull;  
end
```

1.1 Experimenting with CFL condition

The CFL number influences the stability of the method. CFL condition:

$$\frac{\Delta t}{\Delta x^2} \leq \frac{1}{2}.$$

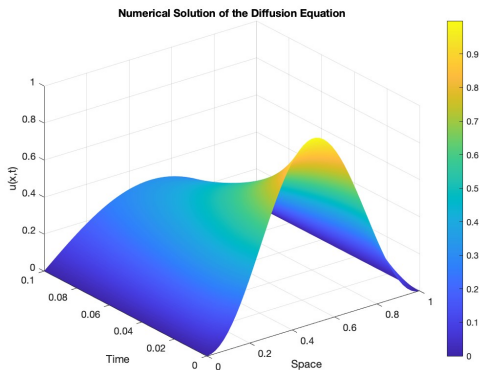
- $N = 50, M = 500 \rightarrow \Delta x = \frac{1}{51}, \frac{\Delta x^2}{2} = 0.00019, \Delta t = 0.0002$, it's slightly above the limit, implying instability.



1.1 Experimenting with CFL condition

CFL condition: $\frac{\Delta t}{\Delta x^2} \leq \frac{1}{2}$.

- $N = 50, M = 600 \rightarrow \Delta x = \frac{1}{51} \rightarrow \Delta t = 0.00016 \rightarrow \text{stability}$



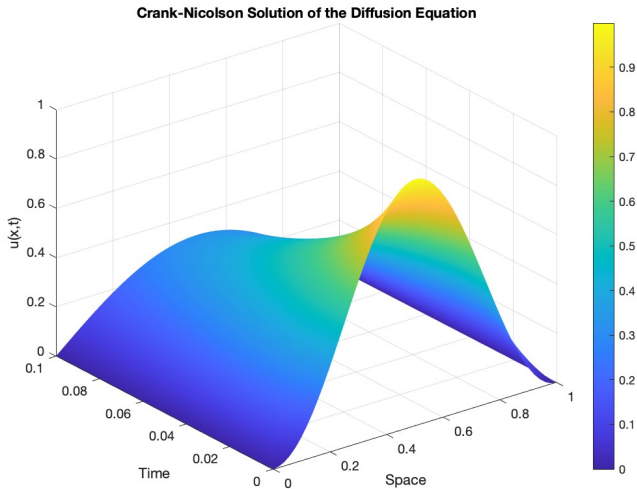
1.2 Crank-Nicolson method

We use A-stable method: trapezoidal rule in time. In each time step we have to solve a linear system of equations

$$\begin{aligned}U^{n+1} &= U^n + \frac{\Delta t}{2}(T_{\Delta x}U^n + T_{\Delta x}U^{n+1}) \\&= (I - \frac{\mu}{2}T)^{-1} \cdot (I + \frac{\mu}{2}T)U^n, \\ \text{where } \mu &= \frac{\Delta t}{\Delta x^2} \text{ and } T = \Delta x^2 T_{\Delta x}\end{aligned}$$

```
function unew = TRstep(Tdx,uold,dt)
N = length(uold);
%I = speye(N); %to create a sparse identity matrix
I = eye(N);
A = I - (dt/2)*Tdx; % left-hand side operator %
B = I + (dt/2)*Tdx; % right-hand side operator %

rhs = B*uold;
unew = A\rhs; % solve the linear system
end
```

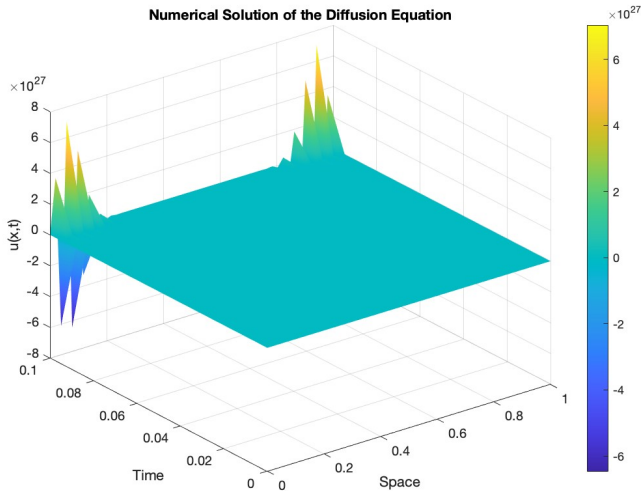


Observations on Crank-Nicolson method

- Stable solutions also where we had instability before due to violated CFL condition.
- We can use much larger time steps Δt and still have stability, without causing the solution to blow up numerically.
- Changing the initial condition $g(x) = x \cdot (1 - x) \rightarrow$ we have still stability regardless of the initial conditions.
- Running the same problem with an explicit Euler method and a large time step $\rightarrow$ instability

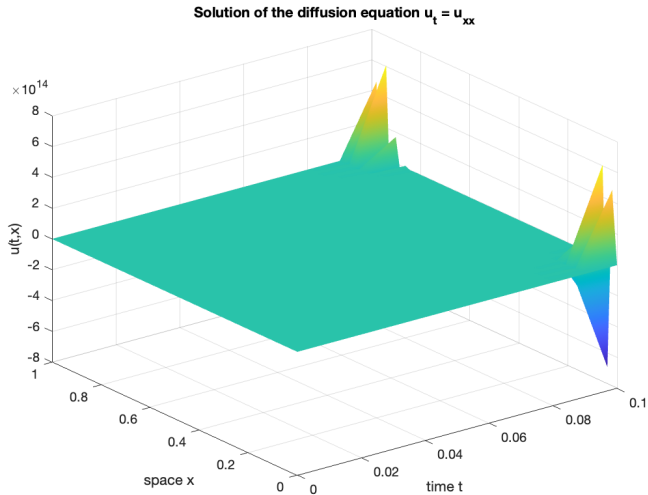
1.2

What we have if we use large time step in Euler Method, same initial condition $g(x) = \sin^2(\pi x)$



1.2

What we have if we change initial conditions in the Euler method with a large time step: $g(x) = x \cdot (1 - x)$



2.1 Advection equation

$$u_t + au_x = 0$$

- Spatial domain $[0, 1]$, N points
- $x_k = (k - 1)\Delta x$ with $k = 1, \dots, N$ and $\Delta x = \frac{1}{N}$
- Periodic boundary conditions on $x \in [0, 1] \rightarrow u(t, 0) = u(t, 1)$
- The solution wraps around from $x = 1$ to $x = 0$ and viceversa.
- The solution at the right boundary, $x_{N+1} = 1$, is u_{N+1} and $u_{N+1} = u_1$ at the left boundary, $x_1 = 0$.
- CFL condition for explicit method is: $a\frac{\Delta t}{\Delta x} \leq C$, $C=1$, constant
- Lax - Wendroff scheme

$$U_j^{n+1} = \frac{a\mu}{2}(1 + a\mu)U_{j-1}^n + (1 - a^2\mu^2)U_j^n - \frac{a\mu}{2}(1 - a\mu)U_{j+1}^n,$$

$$\text{where } a\mu = a\frac{\Delta t}{\Delta x}$$

```

function unew = LaxWen(u, amu)

N = length(u);
unew = zeros(size(u));

% Periodic boundary indices

um = [u(end); u(1:end-1)]; % u_{j-1} with periodic wrap-around
up = [u(2:end); u(1)];      % u_{j+1} with periodic wrap-around

% Apply the Lax-Wendroff formula:

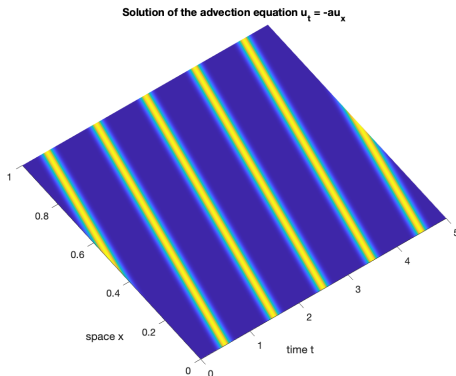
unew = (amu/2)*(1+amu)*um+(1-amu^2)*u-(amu/2)*(1-amu)*up;

end

```

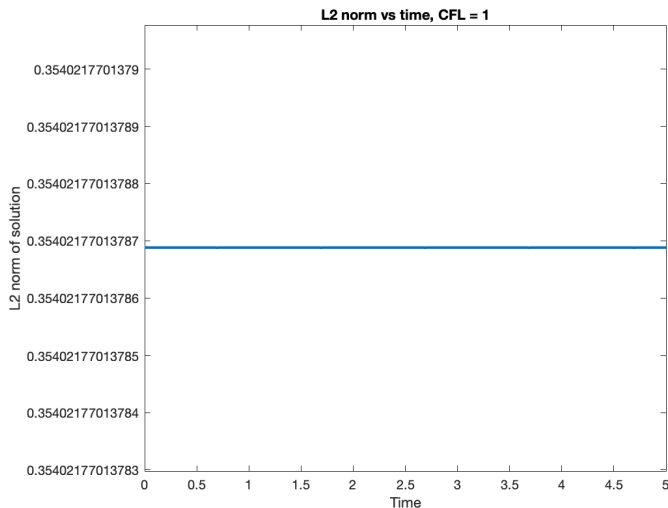
2.1

- $CFL = a \cdot \frac{\Delta t}{\Delta x} = 1$
- $a > 0, a = 1$
- For the advection equation, the characteristics are the straight lines $x - a \cdot t = C$.
- Every line has a slope 'a'.
- The last line wraps around to the first



2.1

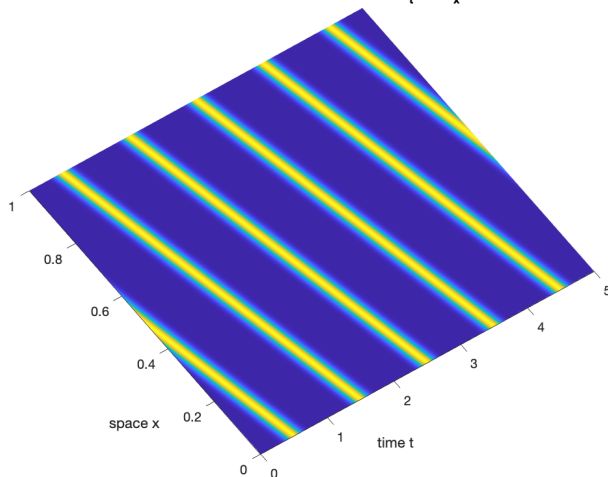
$$CFL = a \cdot \frac{\Delta t}{\Delta x} = 1, \quad a > 0, \quad a = 1$$



2.1

$$CFL = a \cdot \frac{\Delta t}{\Delta x} = 1, \quad a < 0 \quad a = -1$$

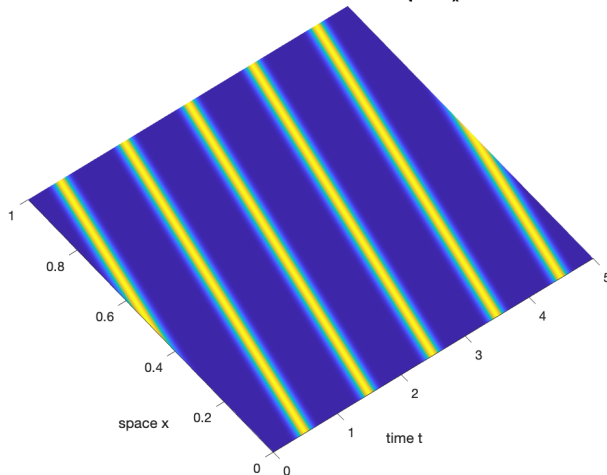
Solution of the advection equation $u_t = -au_x$



2.1

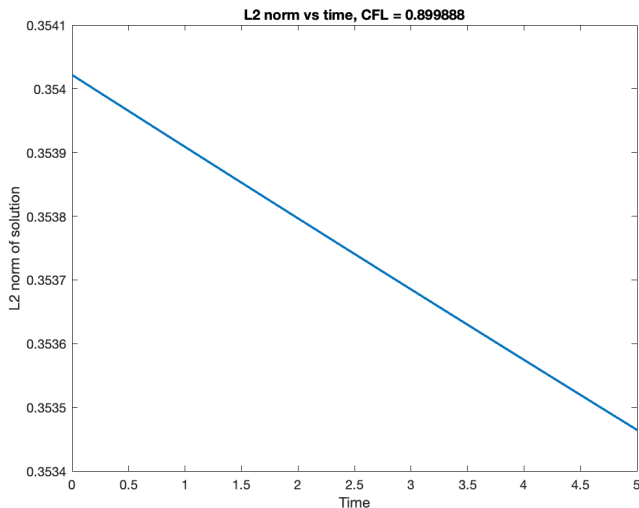
$$CFL = a \cdot \frac{\Delta t}{\Delta x} = 0.9, \quad a > 0$$

Solution of the advection equation $u_t = -au_x$



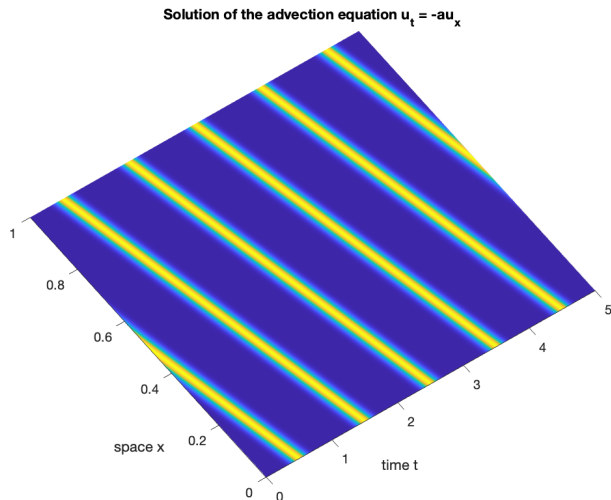
2.1

$$CFL = a \cdot \frac{\Delta t}{\Delta x} = 0.9, \quad a > 0$$



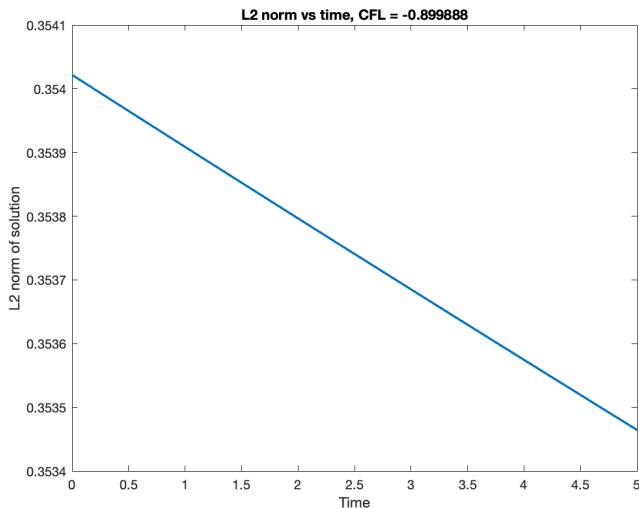
2.1

$$CFL = a \cdot \frac{\Delta t}{\Delta x} = 0.9, \quad a < 0$$



2.1

$$CFL = a \cdot \frac{\Delta t}{\Delta x} = 0.9, a < 0$$



Observations

- $a \frac{\Delta t}{\Delta x} = 1$ the L^2 norm remains constant.
- $a \frac{\Delta t}{\Delta x} = 0.9$ it's below the limit, the L^2 norm (RMS norm) slowly decreases over time, indicating numerical damping.
→ In 2D plots:
- $a > 0$ the pulse moves to the right
- $a < 0$ the pulse moves to the left.
→ In 3D plots:
- We can notice how the wrapping differ from $a > 0$ to $a < 0$.

3. The convection - diffusion equation

$$u_t + a \cdot u_x = d \cdot u_{xx}$$

- a : convection velocity
- d : diffusivity
- $[0, 1]$: spatial domain
- $\Delta x = \frac{1}{N}$
- Initial conditions: $u(0, x) = g(x) \rightarrow g(0) = g(1)$ and $g'(0) = g'(1)$
- Periodic boundary conditions: $u(t, 0) = u(t, 1) \rightarrow$ leads to circulant matrix $\rightarrow$ the rows of the matrix wrap around
- Standard second order difference quotients:

$$\dot{U} = (d \cdot T_{\Delta x} - a \cdot S_{\Delta x})U.$$

```

function unew = convdif(u,a,d,dt,dx)

    N = length(u);
    % First derivative operator (central difference):
    % u_x(i) = (u(i+1)-u(i-1))/(2*dx)
    e = ones(N,1);

    Sdx = (1/(2*dx)) * ( diag(e(1:end-1), -1) - diag(e(1:end-1), 1) );

    % Fix periodic BC for Sx in order to have circulant matrix
    Sdx(1,end) = 1/(2*dx);
    Sdx(end,1) = -1/(2*dx);

    % Second derivative operator:
    % u_xx(i) = (u(i+1)-2*u(i)+u(i-1))/(dx^2)
    Tdx = (diag(-2*e,0) + diag(e(1:end-1),1) + diag(e(1:end-1),-1)) / (dx^2);

    % Fix periodic BC for Tdx
    Tdx(1,end) = 1/dx^2;
    Tdx(end,1) = 1/dx^2;

    L = d*Tdx - a*Sdx;

    I=eye(N);

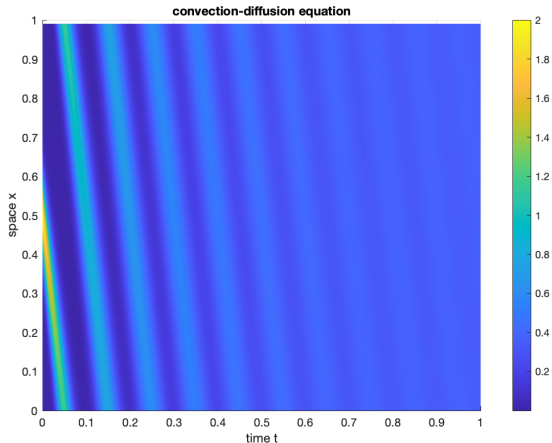
    A = I - 0.5*dt*L; % matrix for U^{n+1}
    B = I + 0.5*dt*L; % matrix times U^n

    rhs = B*u;
    unew = A\rhs; % solve linear system for U^{n+1}
end

```

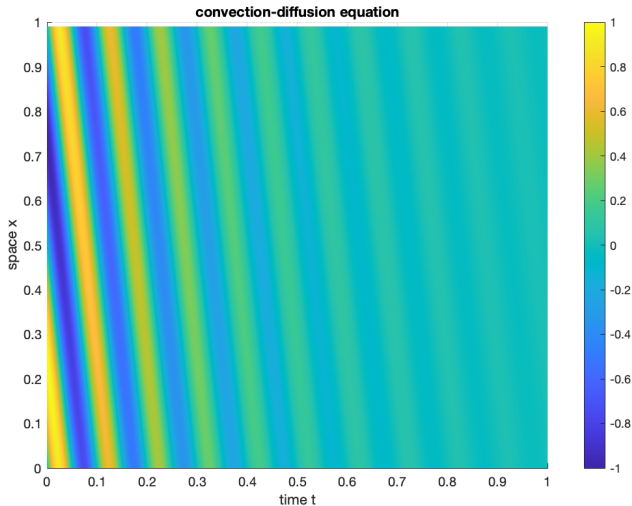
3. Pulse

$$g(x) = e^{-100 \cdot (x-0.5)^2}$$

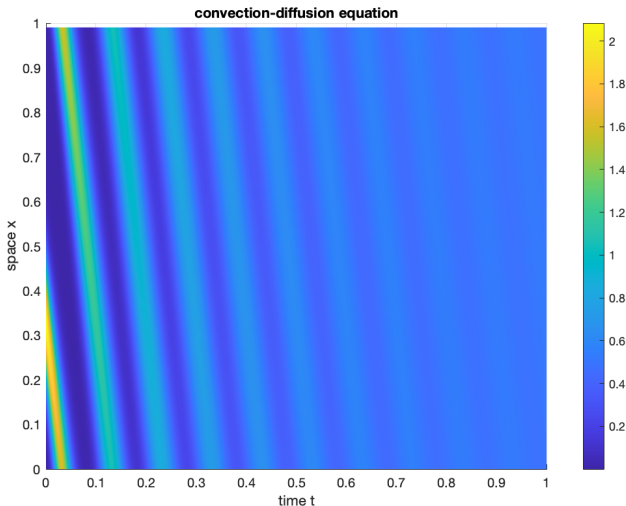


3.

$$g(x) = \sin(2\pi x)$$



A less symmetric function: $g(x) = 2 \cdot e^{\frac{-(x-0.3)^2}{0.02}} \cdot (1 + 3(x - 0.3))$, i.e. we take a component with Gaussian distribution and one linear term.



- For symmetric functions, the peaks are uniform where the bands seems to smoother over time.
- For the less symmetric function, it is possible to notice that the bands are less parallel. Peaks are not uniform as the wave shifts and we have a slower diffusion: the oscillations persist longer.

4. The viscous Burgers equation

$$u_t + uu_x = d \cdot u_{xx}$$

- $\Delta x = \frac{1}{N}$
- Inviscid Burgers equation: $u_t = -uu_x$
- $u(t + \Delta t, x) \approx u(t, x) + \Delta t u_t + \frac{\Delta t^2}{2!} u_{tt}$
- Lax-Wendroff discretization: Differentiated the rhs of $u_t = -uu_x$ with respect to t , to get $u_{tt} = 2u(u_x)^2 + u^2 u_{xx}$ after required substitutions and computation.

```

function u_new = lax_wendroff_step2(u, dx, dt)
    N = length(u);

    % compute first and second derivatives with periodic BC

    ux = (circshift(u, -1) - circshift(u, 1)) / (2*dx);
    uxx = (circshift(u, -1) - 2*u + circshift(u, 1)) / (dx^2);

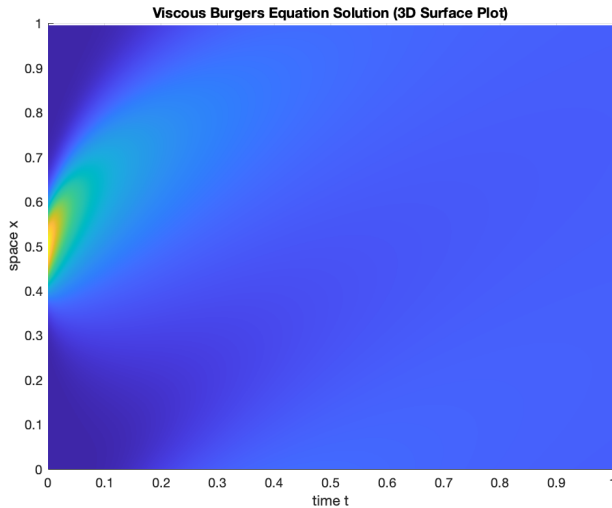
    % compute u_t = -u u_x
    ut = -u .* ux;

    % compute u_tt = 2 u (u_x)^2 + u^2 u_{xx}
    % (u_x)^2 = ux.^2
    utt = 2*u.*(ux.^2) + (u.^2).*uxx; %what we found after the computations

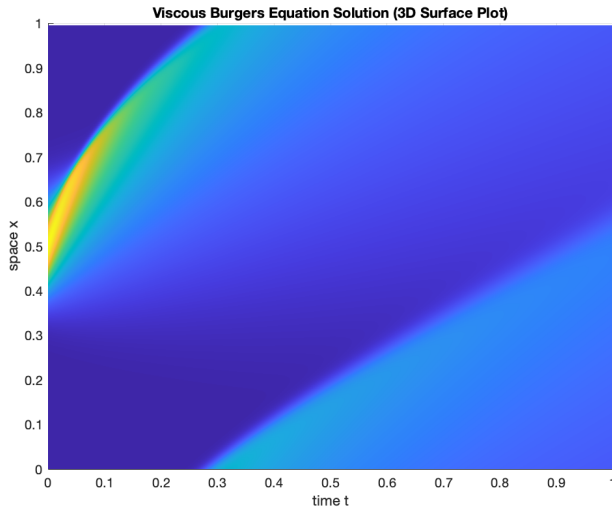
    % Lax-Wendroff update
    u_new = u + dt*ut + 0.5*(dt^2)*utt;
end

```

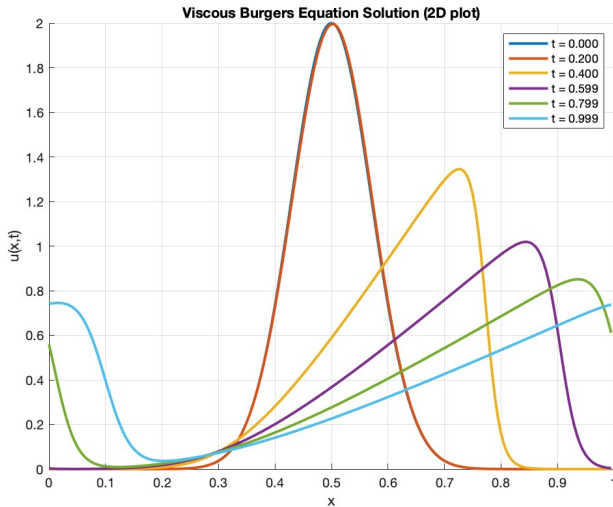
4. $d = 0.1$



4. $d = 0.01$



4. $d = 0.01$



- There is a peak that smooths overtime
- The solution for $d = 0.1$ becomes flat over the t-axis, consequentially it stabilizes over time.
- For $d = 0.01$ we notice that decreasing d reduce the smooth effect.